



Abschlussprüfung Winter 2025

Fachinformatiker Fachrichtung Anwendungsentwicklung

Dokumentation zur betrieblichen Projektarbeit

GAP-Prämienrechner

**Entwicklung eines Rechners für die Berechnung
der Direktzahlung in der Online-Antragstellung**

Abgabedatum: Erfurt, den 26.11.2025

Prüfungsbewerber:

Mykola Havrysh

Am alten Backhaus 5

99098 Erfurt

Prüflingsnummer (145)-41065

Praktikumsbetrieb:

IBYKUS AG für Informationstechnologie

Herman-Hollerith-Straße 1

99099 Erfurt

Ausbildungsbetrieb:

IAD GmbH Erfurt

Maximilian-Welsch-Straße 2a

99084 Erfurt

Inhaltsverzeichnis

1.	Einleitung	1
1.1	Projektfeld	1
1.2	Projektziel	1
1.3	Projektbegründung	2
1.4	Projektabgrenzung	2
2.	Projektplanung	2
2.1	Projektphasen.....	2
2.2	Ressourcenplanung.....	3
2.3	Vorgehensmodell.....	3
3.	Analysephase.....	4
3.1	Ist-/Soll-Analyse	4
3.1.1	Ist-Zustand	4
3.1.2	Soll-Zustand	4
3.2	Qualitätsanforderungen.....	5
3.3	Wirtschaftlichkeitsanalyse.....	5
3.3.1	Projektkosten	5
3.3.2	Nutzwertanalyse	6
3.3.3	Make or Buy-Entscheidung	7
4.	Entwurf.....	7
4.1	Zielplattform und verwendete Ressourcen	7
4.2	Architekturüberblick.....	8
4.3	Excel-Integration und Datenanalyse.....	8
4.4	Entwurf der Geschäftslogik.....	8
4.5	Entwurf der Benutzeroberfläche.....	9
5.	Implementierung.....	9
5.1	Implementierung der Geschäftslogik	9
5.2	Implementierung der Benutzeroberfläche.....	10

5.3	Qualitätssicherung und Test.....	10
6.	Abnahmephase.....	11
7.	Dokumentation.....	11
8.	Fazit.....	11
8.1	Soll-/Ist-Vergleich	11
8.2	Lessons learned.....	12
8.3	Vorstellung beim Kunden und Anbindung in das Produktivsystem der OAS.....	12
	Glossar.....	13
	Abkürzungsverzeichnis.....	14
	Anlagenverzeichnis.....	15
	Quellenverzeichnis.....	32
	Eidesstattliche Erklärung.....	33

Abbildungsverzeichnis

Abbildung 1: Verwendetes Vorgehensmodell.....	22
Abbildung 2: Jira Tickets.....	22
Abbildung 3: Screenshot Excel-Rechner.....	23
Abbildung 4: BasispraemieRepository-Klasse.....	24
Abbildung 5: Sequenzdiagramm.....	25
Abbildung 6: Programmablaufplan.....	26
Abbildung 7: BasispraemieRechnerBean-Klasse.....	27
Abbildung 8: BasispraemieRechnung-Klasse.....	28
Abbildung 9: Klassendiagramm.....	29
Abbildung 10: Screenshot Disclaimer.....	30
Abbildung 11: Screenshot Ergebnis-Seite.....	30
Abbildung 12: CSS-Datei.....	31
Abbildung 13: Screenshot Entwicklerdokumentation Confluence.....	31

Tabellenverzeichnis

Tabelle 1: Grobe Zeitplanung.....	3
Tabelle 2: Projektkosten.....	5
Tabelle 3: Nutzwertanalyse GAP-Rechner.....	6
Tabelle 4: Detaillierte Zeitplanung.....	20
Tabelle 5: Detaillierte Ressourcenplanung.....	21

1 Einleitung

Die vorliegende Projektdokumentation stellt das Abschlussprojekt des Autors im Rahmen seiner Ausbildung zum Fachinformatiker für Anwendungsentwicklung dar.

1.1 Projektumfeld

Der Betrieb, in dem das Projekt durchgeführt wird, ist die IBYKUS AG für Informationstechnologie, die zu den führenden deutschen Software- und Technologieunternehmen gehört.

1990 in Thüringen gegründet, unterhält das IT-Unternehmen mit Hauptsitz in Erfurt heute insgesamt sechs Standorte in Deutschland. Bundesweit arbeiten über 370 Mitarbeitende tagtäglich daran, sowohl Unternehmen als auch die öffentliche Verwaltung mit verschiedensten Softwarelösungen und intelligenten Technologien einfacher, sicherer und digitaler zu gestalten.

Das Leistungsspektrum reicht von umfassenden E-Government-Lösungen für die Verwaltung europäischer, nationaler und kommunaler Fördermittel bis hin zu innovativen Lösungen zur Prozessdigitalisierung für Unternehmen und Behörden.

Die Entwicklung mobiler Anwendungen, IT-Security, IT-Infrastruktur sowie der zuverlässige Betrieb von Lösungen in eigenen BSI-zertifizierten Rechenzentren in Deutschland und eine professionelle Beratung runden das Portfolio ab.

Das wichtigste Geschäftsfeld ist die Fördermittelverwaltung für landwirtschaftliche Fördermittel der EU. Dafür gibt es das Langzeitprojekt der Online-Antragstellung (OAS), über das Antragsteller des Landes Hessen ihre Fördermittelanträge stellen können.

Der GAP-Prämienrechner wird als Teil der OAS geplant, deren Auftraggeber die Wirtschafts- und Infrastrukturbank in Hessen ist, welche für die landwirtschaftliche Förderung in Ihrem Bundesland zuständig ist.

1.2 Projektziel

Das Ziel des Projekts ist die Entwicklung und Integration eines GAP-Prämienrechners (Berechnungstool für EU-Agrarförderungen) in die OAS. Damit können landwirtschaftliche Betriebe auf Basis ihrer Flächen- und Tierdaten einfach eine Prognose über ihre voraussichtlichen Direktzahlungen erhalten.

Der Rechner soll zuvor eingegebene Formulardaten des Antragstellers zur Berechnung aus der Datenbank auslesen. Hierzu gehören die Flächendaten, der aktuelle Tierbestand und die allgemeinen Betriebsdaten des Antragstellers. Anschließend soll dem Antragsteller klar und übersichtlich die prognostizierte Prämie angezeigt werden. Um Transparenz und Nachvollziehbarkeit der Ergebnisse zu gewährleisten, werden zusätzlich alle relevanten Betriebsdaten angezeigt, welche für die Berechnung herangezogen wurden.

Das Ziel ist erreicht, wenn es dem Projektleiter vorgestellt und abgenommen wurde.

1.3 Projektbegründung

In der Praxis sind Antragsteller bislang gezwungen, eine Excel-Vorlage herunterzuladen und ihre Daten manuell einzutragen, um vorab eine Prognose über mögliche Zahlungen zu erhalten. Dabei handelt es sich jedoch um Informationen, die ohnehin bereits vollständig in der OAS für die Antragstellung erfasst werden. Dies führt zu erheblichem Zeitaufwand und redundanter Dateneingabe, insbesondere wenn Angaben aus unterschiedlichen Quellen erneut zusammengetragen werden müssen. Zudem entstehen regelmäßig Fehler (Tippfehler, vergessene Angaben etc.), die zu fehlerhaften Berechnungen oder Rückfragen führen. Um das zu vermeiden, besteht der Bedarf an einem in die OAS integrierten Rechner, der diese manuellen Schritte automatisiert.

1.4 Projektabgrenzung

Das Projekt umfasst die Entwicklung eines webbasierten Prämienrechners, der automatisch gespeicherte Betriebs-, Flächen- und Tierdaten ausliest und damit Förderprämien berechnet. Aus diesem Grund sind nicht Teil des Projektes die umfangreichen manuellen Dateneingaben, Bearbeitung oder Änderung der Daten im System. Außerdem ist automatisierte Testentwicklung (Unit-Tests, Datenvalidierung) auch nicht Teil des Projektes, da es keine Benutzereingabe gibt und der Vergleich zwischen dem Ergebnis des Online-Prämienrechners und dem bestehenden Excel-Rechner als Maßstab für die Korrektheit dient.

2 Projektplanung

Im Folgenden wird die Projektplanung beschrieben.

2.1 Projektphasen

Für das Projekt ist ein Gesamtaufwand von 80 Stunden vorgesehen. Vor dem Start des Projekts wurde es in verschiedene Phasen unterteilt. Diese umfassen eine Phase der Analyse und Anforderungsermittlung, in der geklärt wird, welche Daten vorhanden sind und wie die Ausgabe erfolgen soll. Darauf folgt die Phase der Realisierung, bei der der Prämienrechner entwickelt wird: Datenbankanbindung, Berechnungslogik und Benutzeroberfläche. Danach wird in der Phase der Verifikation die Korrektheit geprüft. Anschließend findet der Projektabschluss durch die Abnahme des Projektleiters der OAS statt, wobei mögliche Anmerkungen aufgenommen und Anpassungen vorgenommen werden können. Zuletzt wird eine Confluence-Seite mit Entwicklerdokumentation erstellt sowie diese Projektdokumentation fertiggestellt.

Eine detaillierte Zeitplanung findet sich im *Anhang A5: Detaillierte Zeitplanung*

Projektphase	Geplante Ziel
Projektplanung	3 Std.
Analyse	12 Std.
Realisierung und Test	40 Std.
Abschluss des Projektes	3 Std.
Schreiben einer Dokumentation	22 Std.
Gesamt	80 Std.

Tabelle 1: Grobe Zeitplanung

2.2 Ressourcenplanung

Für die Umsetzung wird das durch den Praktikumsbetrieb gestellte Notebook mit Windows 11, ein Büroarbeitsplatz mit zwei weiteren Monitoren, sowie einer Tastatur, einem Headset und einer Maus verwendet.

Softwareseitig wurde IntelliJ IDEA Community Edition als Entwicklungsumgebung verwendet. Die OAS-Umgebung läuft auf einem lokalen Tomcat-Server. Für Testdaten wird zusätzlich eine lokale PostgreSQL-Datenbank verwendet.

Weiterhin wurde ein Git-Repository auf BitBucket angelegt, um den Fortschritt des Projektes zu sichern und anderen Entwicklern zugänglich zu machen. Zur Kommunikation mit dem Projektleiter sowie mit anderen OAS-Entwicklern wurden Messengerdienste wie Teams, Slack und E-Mail verwendet.

Eine detaillierte Auflistung der verwendeten Ressourcen ist unter *Anhang A6: Detaillierte Ressourcenplanung* zu finden.

2.3 Vorgehensmodell

Üblicherweise wird im Projekt der OAS im Team die SCRUM-Methode verwendet. Für das vorliegende Projekt ist dieses Vorgehen jedoch nicht geeignet: Die Entwicklung wird ausschließlich von einer einzelnen Person durchgeführt, und die zeitlichen Vorgaben sind klar definiert. Damit passt SCRUM-Modell nicht zu den Rahmenbedingungen des vorliegenden Projekts.

Für die Durchführung wurde sich für das erweiterte Wasserfallmodell entschieden, siehe *Anhang A7: Verwendetes Vorgehensmodell*

Dieses Modell sieht vor, die einzelnen Projektphasen nacheinander abzuarbeiten. Gleichzeitig besteht die Möglichkeit bei Bedarf, einen Schritt zurückzugehen, um notwendige Anpassungen vorzunehmen, ohne das gesamte Projekt zu beenden. Zur Unterstützung der Projektorganisation

wurde ein Jira-Projekt eingerichtet. Mit diesem Arbeitsmittel können die offenen Aufgaben und der Projektfortschritt jederzeit im Blick behalten werden. Gleichzeitig ist der Stand des Projekts für andere nachvollziehbar und dokumentiert. Im Anhang A8: Jira Tickets zum GAP-Rechner sind die angelegten Jira-Tickets dargestellt.

3 Analysephase

3.1 Ist-/Soll-Analyse

3.1.1 Ist-Zustand

Zurzeit haben Antragsteller, die sich über die Summe möglichen Direktzahlungen informieren möchten, keine direkte Online-Möglichkeit. Stattdessen müssen sie eine Excel-Datei herunterladen und lokal auf ihrem Rechner öffnen. Der Screenshot der Excel-Tabelle ist unter Anhang A9: Screenshot Excel-Rechner zu sehen.

Dort tragen die Antragsteller ihre betrieblichen Daten manuell ein, zum Beispiel Flächenangaben oder Angaben zur Tierhaltung. Erst nach dieser Eingabe zeigt die Excel-Tabelle die voraussichtlichen Prämien.

Da viele der notwendigen Daten bereits in der OAS-Datenbank gespeichert sind, bestehen alle notwendigen technischen Möglichkeiten, den Antragstellern ein Online-Werkzeug zur Verfügung zu stellen. Dieses soll die vorhandenen Daten automatisch einlesen und die Ergebnisse übersichtlich und klar anzeigen.

3.1.2 Soll-Zustand

Im Soll-Zustand soll der Prämienrechner die Berechnung der möglichen Direktzahlungen für den Antragsteller weitgehend automatisieren. Ein manueller Eingabeschritt einzelner Daten ist grundsätzlich vorgesehen, ist auf dieser Stufe jedoch nicht erforderlich. Das System greift direkt auf die in der Datenbank gespeicherten Informationen zu, die der Antragsteller bereits zuvor eingegeben hat. Nach dem Laden dieser Daten berechnet die Anwendung automatisch die Basisprämie und weitere Prämienbestandteile. Die Ergebnisse werden dem Benutzer in einer übersichtlichen und klar strukturierten Oberfläche dargestellt. Das System verfolgt damit drei Hauptziele: Zum einen werden mögliche Eingabefehler vermieden, da keine zusätzliche manuelle Übertragung von Werten notwendig ist. Zum anderen erhöht sich die Benutzerfreundlichkeit: viele Antragsteller, die bisher die Excel-Variante nicht genutzt haben, erhalten nun die Möglichkeit, direkt einzusehen, welche Zahlungen sie voraussichtlich erhalten werden. Drittens führt die Nutzung des Prämienrechners zu einer erheblichen Zeitersparnis, da die Antragsteller keine Excel-Tabelle mehr manuell ausfüllen müssen.

3.2 Qualitätsanforderungen

Für den Prämienrechner gelten bestimmte Qualitätskriterien. Diese sollen sicherstellen, dass das System zuverlässig, benutzerfreundlich und verständlich ist:

- **Zuverlässigkeit und Genauigkeit:** Die Berechnungen müssen korrekt und stabil durchgeführt werden. Das Ergebnis soll jederzeit mit dem bisherigen Excel-Kalkulator übereinstimmen.
- **Benutzerfreundlichkeit:** Die Oberfläche muss klar und übersichtlich gestaltet sein. Der Nutzer soll ohne komplexe Eingaben sofort die relevanten Informationen sehen.

3.3 Wirtschaftlichkeitsanalyse

3.3.1 Projektkosten

Nachfolgend werden die Projektkosten aufgelistet. In der Rechnung werden sowohl die Personalkosten aller involvierten Mitarbeiter als auch sonstige Kosten, wie Arbeitsplatz und Strom, beachtet. Die Berechnung wird in die verschiedenen Projektphasen Planung, Analyse, Realisierung und Test, Abschluss und Dokumentation eingeteilt.

Als Personalkosten wurden Stundenpauschalen von 10 € für einen Auszubildenden, 30 € für einen Softwareentwickler und 50 € für einen Projektleiter angesetzt. Außerdem wird eine stündliche Kostenpauschale von 5 € pro Stunde für sonstige Kosten zur Berechnung genutzt. Final ergeben sich daraus Projektkosten von 1.380,00 €. Bei den Personalkosten handelt es sich um fiktive Werte, da die tatsächlichen Beträge aus Gründen der Geheimhaltung nicht veröffentlicht werden dürfen.

Vorgang	Mitarbeiter	Zeit	Personal	Sonstige Kosten	Gesamt
Planung	1x Auszubildender	3 h	30 €	15 €	45 €
Analyse	1x Auszubildender	12 h	120 €	60 €	180 €
Realisierung	1x Auszubildender	38 h	380 €	190 €	570 €
Qualitätssicherung	1x Auszubildender	4 h	40 €	20 €	60 €
Dokumentation	1x Auszubildender	22 h	220 €	110 €	330 €
Fachgespräch	1x Softwareentwickler	2 h	60 €	10 €	70 €
Code-Review	1x Softwareentwickler	2 h	60 €	10 €	70 €
Abnahme	1x Projektleiter	1 h	50 €	5 €	55 €
Gesamt					1.380 €

Tabelle 2: Projektkosten

3.3.2 Nutzwertanalyse

Im Folgenden wird eine Gegenüberstellung der Aufwände und Vorteile vorgenommen, um zu prüfen, ob die Entwicklung des GAP-Prämienrechners sinnvoll ist oder ob weiterhin ausschließlich das Excel-Dokument genutzt werden sollte.

Ein zentrales Kriterium ist dabei der Zeitaufwand. Viele Antragsteller verfügen über eine große Anzahl an landwirtschaftlichen Flächen und Tieren. Da Zeit im landwirtschaftlichen Betrieb ein wesentlicher Faktor ist, möchten Antragsteller möglichst schnell erfahren, in welchem Maße ihre Flächen förderfähig sind. In einem Excel-Dokument müssen zunächst alle Daten manuell eingegeben werden, bevor die Ergebnisse sichtbar sind. Bei einem Online-Prämienrechner hingegen wird das Ergebnis ohne Wartezeit berechnet.

Der nächste wichtige Faktor ist die Fehleranfälligkeit der Ergebnisse. Beim Excel-Dokument können Eingabefehler leicht auftreten, die das Resultat verfälschen. Der GAP-Prämienrechner bietet eine höhere Sicherheit, da die Daten direkt aus dem System übernommen werden.

Ein weiterer wesentlicher Faktor ist die Benutzerfreundlichkeit. Die Notwendigkeit, die Excel-Datei herunterzuladen und zwischen dieser und dem OAS-Portal hin- und herzuwechseln, stellt ebenfalls eine erhebliche Umständlichkeit dar.

Darüber hinaus spielt auch die Erweiterbarkeit eine Rolle. Einmal entwickelt, kann der Online-Prämienrechner künftig leicht an neue Berechnungslogiken und veränderte Anforderungen angepasst werden. Im Gegensatz dazu muss beim Excel-Dokument der Antragsteller selbst darauf achten, stets die aktuelle Version zu finden, herunterzuladen und zu verwenden.

Die Tatsache, dass der Excel-Prämienrechner den Antragstellern bereits zur Verfügung steht und die Entwicklung eines Online-Prämienrechners zunächst finanzielle Aufwände erfordert, gehört zu den Nachteilen. Die Nutzung eines Online-Rechners bietet jedoch langfristige finanzielle Vorteile, was sicherlich für seine Entwicklung spricht.

Bewertungskriterien	Gewichtung	GAP-Prämienrechner	Excel Dokument
Zeitaufwand der Benutzung	0,30	5	1
Fehleranfälligkeit	0,25	5	3
Benutzerfreundlichkeit	0,05	5	3
Erweiterbarkeit	0,10	3	3
Kosten	0,30	4	5
Summe	1	4,5	3,0

Tabelle 3: Nutzwertanalyse GAP-Rechner

Aus der Nutzwertanalyse ergibt sich, dass die Vorteile des Online-Rechners den Nachteilen überwiegen. Das Projekt bietet enorme Vorteile, insbesondere im Hinblick auf die Zeitersparnis, die Fehlervermeidung und finanziellen Nutzen.

3.3.3 Make or Buy-Entscheidung

Eine externe Lösung in Form des Excel-Dokuments ist bereits vorhanden und seit Jahren im Einsatz. Sie erfordert jedoch, dass Antragsteller die Datei herunterladen, manuell ausfüllen und anschließend die Ergebnisse interpretieren. Diese Vorgehensweise ist fehleranfällig, zeitaufwendig und benutzerunfreundlich, da die Daten nicht automatisch aus dem OAS-System übernommen werden.

Dagegen bietet die Eigenentwicklung des GAP-Prämienrechners innerhalb des OAS-Portals („Make“) entscheidende Vorteile: Automatisierung, Fehlerfreiheit, Benutzerfreundlichkeit und Transparenz. Die Nachteile der Eigenentwicklung liegen hauptsächlich in den initialen Entwicklungsaufwänden. Diese Faktoren sind jedoch angesichts der langfristigen Einsparungen an Zeit und Fehlervermeidung sowie der höheren Nutzerfreundlichkeit von untergeordneter Bedeutung. Zudem soll die Funktionalität ausschließlich im OAS-Portal bereitgestellt werden; ein externer Zugriff ist nicht vorgesehen. Vor diesem Hintergrund scheidet eine externe Lösung („Buy“) aus.

Daraus folgt, dass der GAP-Prämienrechner vom Autor neu entwickelt werden sollte.

4 Entwurf

Im Folgenden werden die Entwürfe für die Umsetzung des Projektes beschrieben.

4.1 Zielplattform und verwendete Ressourcen

Die Anwendung wird im bestehenden OAS-Portal integriert und läuft in einer Java-EE-Umgebung auf einem Tomcat-Server. Als Programmiersprache wird Java verwendet, das Benutzer-Interface wird mit JSF (Jakarta Faces) umgesetzt und mittels CSS wird das Aussehen angepasst. Für die Verarbeitung der Daten kommt der JSON-Parser von „org.json“ zum Einsatz. Die Daten werden aus der PostgreSQL-Datenbank der OAS entnommen. Die verwendete Entwicklungsumgebung ist IntelliJ IDEA. Das Build- und Abhängigkeitsmanagement erfolgt über Maven. Die Versionsverwaltung geschieht über BitBucket. Die Projektorganisation und Dokumentation wird in Jira gepflegt.

Damit basiert die Zielplattform vollständig auf bewährten und im Unternehmen bereits eingesetzten Technologien und integriert sich nahtlos in die vorhandene Infrastruktur.

4.2 Architekturüberblick

Die Architektur des Projekts zeigt eine Schichtung in Datenzugriff, Geschäftslogik und Präsentation und sorgt dafür, dass Datenbeschaffung, Berechnung und Darstellung klar voneinander getrennt sind. Im Zentrum steht die Bean-Klasse, die als Bindeglied zwischen Benutzeroberfläche und Geschäftslogik fungiert. Sie ist für die Steuerung des Ablaufs verantwortlich, ruft die benötigten Daten über das Repository ab und übergibt diese an die Berechnungskomponente. Das Repository stellt Methoden bereit, um Flächen-, Tierdaten und weitere relevante Informationen strukturiert zu laden.

Die eigentliche Berechnungslogik ist in der Rechnung-Klasse gekapselt, die die Berechnungen zur Ermittlung der Prämien durchführt. Die Ergebnisse werden anschließend in der Ergebnis-Klasse gesammelt, die als Datencontainer fungiert und sämtliche Ergebnisse verfügbar macht.

4.3 Excel-Integration und Datenanalyse

Die fachliche Logik des Prämienrechners wurde aus einer bestehenden Excel-Vorlage manuell abgeleitet. Dabei wurden die relevanten Tabellenblätter, Formeln und Parameter identifiziert, die Rechenschritte nachvollzogen und in klare Rechenregeln überführt. Auf Basis dieser Herleitung wurden die benötigten Eingangsdaten den vorhandenen Datenbanktabellen zugeordnet.

4.4 Entwurf der Geschäftslogik

Grundlage bildet dabei das BasispraemieRepository, das alle notwendigen Informationen zu Flächen, Tierhaltung und betrieblichen Angaben direkt aus der PostgreSQL-Datenbank lädt (Siehe Codeausschnitt im Anhang A10: Ausschnitt aus der BasispraemieRepository-Klasse). Auf dieser Basis greift die eigentliche Berechnungsschicht, bestehend aus den Klassen BasispraemieRechnung und BasispraemieAggregatInfo, die die Daten anhand festgelegter Fördersätze und Regeln weiterverarbeitet. Das Ergebnis dieser Logik ist ein Objekt des Typen BasispraemieResult, das sämtliche Zwischenergebnisse und Summen enthält. Die Anbindung an die Benutzeroberfläche erfolgt schließlich über die Bean BasisPraemieRechnerBean, die die Daten aus dem Repository abrufen, an die Berechnungsschicht übergibt und die Ergebnisse für die Darstellung in der JSF-Oberfläche bereitstellt. Gleichzeitig übernimmt sie die Steuerung für Benutzereingaben, wie zum Beispiel die Bestätigung des Disclaimers oder zusätzliche Eingabefelder. Durch diese klare Trennung zwischen Datenhaltung, Berechnung und Präsentation bleibt die Architektur des Systems übersichtlich, erweiterbar und gut wartbar.

Eine visuelle Darstellung der Abläufe und Zusammenhänge befindet sich im Anhang A11: Sequenzdiagramm sowie im Anhang A12: Programmablaufplan.

4.5 Entwurf der Benutzeroberfläche

Die Benutzeroberfläche des Prämienrechners wird mit JSF und XHTML umgesetzt, die innerhalb des bestehenden OAS-Portals eingebunden sind. Der Aufbau orientiert sich an folgender Struktur: Zunächst wird dem Benutzer ein Hinweistext in Form eines Disclaimers angezeigt, den er bestätigen muss, bevor die eigentlichen Funktionen des Rechners zugänglich sind. Dadurch wird sichergestellt, dass der Nutzer die notwendigen Informationen zur Kenntnis nimmt.

Nach Bestätigung des Disclaimers öffnet sich die Hauptseite des Rechners. Hier werden die Ergebnisse der Berechnung in einer eigenen Tabelle dargestellt, die die berechneten Prämienbestandteile für das Jahr 2025 übersichtlich nach Position, Rate pro Einheit und Gesamtsumme auflistet. Die relevanten Daten des Betriebes wie Betriebsname, Wirtschaftsform, Junglandwirt-Status und Angaben zur Milchabgabe werden in einer eigenen Tabelle angezeigt. Darauf folgt eine kompakte Übersicht über die landwirtschaftlichen Flächen (Ackerland, Grünland, Dauerkulturen) sowie die Gesamtsumme der förderfähigen Flächen. Ein weiterer Abschnitt der Oberfläche enthält eine Tabelle mit Daten zu Mutterkühen, Schafen und Ziegen.

Die Gestaltung wird durch eine CSS-Datei umgesetzt.

5 Implementierung

Im Folgenden wird die Entwicklung und Implementierung des Projekts beschrieben.

5.1 Implementierung der Geschäftslogik

Die Geschäftslogik setzt sich aus drei Hauptaufgaben zusammen: die Datenbeschaffung aus der OAS-Datenbank, die fachliche Aufbereitung und die eigentliche Prämienberechnung. Der Ablauf erfolgt zentral durch die View-Bean `BasisPraemieRechnerBean`; sie verknüpft Repository, Rechenkassen und Benutzeroberfläche. Beim Laden der Seite ruft `BasisPraemieRechnerBean.init()` zunächst den Personen-Ident aus der Session (`sessionBean.getOasxUser().getPI()`) ab und stößt dann die Datenermittlung im `BasispraemieRepository` an. Dort bereiten die Methoden `getAF(pi)`, `getDGL(pi)` und `getDK(pi)` die Informationen zu Flächen, sowie die Methode `getPersonInfoById(pi)` die persönlichen Daten auf. Die folgenden booleschen Variablen werden aus JSON-Feldern der GA-Tabellen gelesen: `isOekologisch(pi)` prüft die Wirtschaftsform, `isJunglandwirt(pi)` prüft den Junglandwirtstatus und `gibtKuhmilchAb(pi)` wertet die Kuhmilchabgabe aus. `getTierData(pi)` liefert Tierzahlen, u. a. Rinder, Schafe, Ziegen, Equiden etc., während `getGesData(pi)` die aggregierten Eingabegrößen für Mutterschafe, Mutterziegen und Mutterkühe bereitstellt. Ein Auszug aus dem Code der Klasse `BasisPraemieRechnerBean` ist im Anhang A13: Ausschnitt aus der `BasispraemieRechnerBean`-Klasse zu sehen.

Sind alle Daten geladen, übergibt die Bean sie an BasispraemieRechnung. Auf dieser Basis bildet diese Klasse den Kern der Rechnungslogik: sie summiert die Flächen, berechnet die Basisprämie, sowie Umverteilungsprämie, Junglandwirteprämie und gekoppelte Tierprämien. Das Ergebnis wird als BasispraemieResult zurückgegeben und in der Bean gespeichert. Für die tabellarische Ausgabe stellt diese Klasse zugleich aufbereitete Zeilen mithilfe der Methode `asTableRows()` bereit, damit die JSF-Seite die Felder „je Einheit“ und „Gesamt“ formatiert darstellen kann. Ein Codeausschnitt aus der Klasse BasispraemieRechnung ist im Anhang A14: Ausschnitt aus der BasispraemieRechnung-Klasse zu sehen.

Darüber hinaus stellt die Klasse BasispraemieAggregatInfo Werte für Berechnungen und Zusatzinformationen bereit, die die Berechnungen unterstützen und verständlich machen.

Eine Übersicht der in diesem Kapitel erwähnten Klassen findet sich im Anhang A15: Klassendiagramm.

5.2 Implementierung der Benutzeroberfläche

Die Benutzeroberfläche des GAP-Prämienrechners wurde mit Java Server Faces realisiert und in das bestehende Portal integriert. Grundlage dafür bildet die Datei `basispraemie-rechner.xhtml`, die auf einem Template basiert und so in einem einheitlichen Schema mit anderen Bereichen des Portals erstellt wird.

Zunächst wird ein Disclaimer angezeigt, der von den Antragstellern bestätigt werden muss (siehe Anhang A16: Screenshot Disclaimer). Erst nach Zustimmung durch einen Checkbox-Eintrag und Klick auf die Schaltfläche „Weiter“ wird der Rechner sichtbar.

Im Hauptbereich der Seite werden die Informationen zum Betrieb dargestellt. Hierfür kommen `h:panelGrid`-Komponenten zum Einsatz, die Name, Anschrift, Wirtschaftsform sowie weitere Merkmale anzeigen. Zusätzlich liefert die Tabelle Informationen zur Flächenangaben (Ackerland, Grünland, Dauerkulturen sowie die gesamte förderfähige Fläche) und die Anzahl der Mutterkühe oder Mutterschafe. Eine Darstellung der Ergebnisse ist im Anhang A17: Screenshot Ergebnis-Seite zu sehen.

Die Gestaltung und das Layout werden über die CSS-Datei `basispraemie.css` gesteuert. Ein Auszug aus dieser Datei ist unter Anhang A18: Ausschnitt aus der CSS-Datei zu finden.

5.3 Qualitätssicherung und Test

Die Qualitätssicherung des Projekts erfolgte durch den Abgleich der Rechenergebnisse zwischen der Excel und dem neuen Rechner. Anhand von Beispieldaten wurden sämtliche Berechnungsschritte überprüft und die Ergebnisse miteinander verglichen. Dadurch konnte sichergestellt werden, dass die Umsetzungen in Java exakt den Vorgaben entsprechen.

Ohne bestehende Komponenten der OAS zu beeinflussen, wurde auch die Benutzeroberfläche getestet. Dabei stand im Vordergrund, dass die Informationen für den Antragsteller korrekt und übersichtlich dargestellt werden.

Auf automatisierte Tests wurde in diesem Projekt bewusst verzichtet. Stattdessen diente der Abgleich mit der Excel als maßgebliches Kriterium für die Richtigkeit. Darüber hinaus wurden manuelle Tests mit unterschiedlichen Betrieben durchgeführt, um verschiedene Szenarien abzudecken, wie etwa Betriebe mit oder ohne große Flächen oder großen Viehbeständen, in Kombination mit unterschiedlichen Werten der Flags „Junglandwirt“ und „Kuhmilchabgabe“.

6 Abnahmephase

Das Projekt wurde lokal entwickelt. In dieser Phase wurde die Anwendung dem Projektleiter vorgestellt und auf die Einhaltung der zuvor festgelegten Anforderungen geprüft. Entscheidend war dabei, dass die berechneten Ergebnisse des GAP-Prämienrechners mit den Resultaten der Excel-Vorlage übereinstimmten. Auch bei der Gestaltung der Benutzeroberfläche wurde Wert darauf gelegt, dass alle notwendigen Informationen wie Flächendaten, Tierzahlen und relevante Prämienbestandteile klar und verständlich dargestellt werden. Dies wurde durch den Projektleiter abgenommen. Somit wurde das Projekt erfolgreich abgeschlossen. Außerdem besteht die Möglichkeit zur Erweiterung und Verbesserung dieser Anwendung, die auf Kundenwunsch umgesetzt wird.

7 Dokumentation

Für die Entwickler wurde eine Dokumentation in Confluence geschrieben, die die erstellten Funktionen in Java, sowie den Aufbau der XHTML erläutert. Dadurch sollen mögliche zukünftige Anpassungen erleichtert werden. Auf eine Anwenderdokumentation wurde verzichtet, da aufgrund der automatischen Rechnung der GAP-Rechner nicht bedient werden muss.

Ein Ausschnitt aus der Entwicklerdokumentation ist im Anhang A19: Screenshot Entwicklerdokumentation Confluence zu sehen.

8. Fazit

Im Folgenden werden die Ergebnisse des Projektes sowie die Erkenntnisse des Autors beschrieben.

8.1 Soll-/Ist-Vergleich

Der Ist-Zustand zeigt, dass diese Anforderungen im Wesentlichen erfüllt wurden. Der entwickelte Rechner liest die relevanten Daten zu Flächen, Tierhaltung und Betriebsinformationen direkt aus der Datenbank aus, führt die Berechnungen gemäß den vorgegebenen Formeln durch und stellt

die Ergebnisse strukturiert und verständlich dar. Auch die Übersichtlichkeit der Oberfläche konnte wie geplant umgesetzt werden, wie die Abnahme bestätigte.

Unterschiede bestehen lediglich im Detail: ein Disclaimer wurde eingeführt, der ursprünglich nicht explizit gefordert war, aber aus rechtlichen Gründen sinnvoll erschien. Insgesamt entspricht das Ergebnis den definierten Zielen und die Anwendung erfüllt den geplanten Zweck in vollem Umfang.

8.2 Lessons learned

Im Verlauf des Projekts konnten wertvolle Erfahrungen gesammelt werden. Besonders deutlich wurde, wie wichtig die Phasen der Analyse, der Planung und der Erstellung eines ersten Entwurfs sind. Dadurch lässt sich von Anfang an eine Projekt-Roadmap entwickeln, die es wesentlich erleichtert, den Code zu implementieren und den Arbeitsverlauf anzupassen.

Auf technischer Ebene konnte der Autor wertvolle Erfahrungen im objektorientierten Programmieren mit Java sammeln. Neu war zudem die Möglichkeit, Java mit XHTML für die Gestaltung der Benutzeroberfläche zu kombinieren.

Ein weiterer zentraler Aspekt war die Dokumentation des Projektfortschritts in Jira. Die Aufteilung des Prozesses in kleinere Teilaufgaben und das Führen täglicher Notizen fördern einen systematischen Ansatz bei der Problemlösung. Zudem ermöglicht es, diese Erfahrungen künftig auch auf größere Projekte zu übertragen.

8.3 Vorstellung beim Kunden und Anbindung in das Produktivsystem der OAS

Der entwickelte Prämienrechner stellt bereits eine erhebliche Erleichterung für die Antragsteller dar. Dennoch bestehen verschiedene Ansatzpunkte für eine zukünftige Weiterentwicklung. Eine wichtige Anforderung ergibt sich aus den sich regelmäßig ändernden Rahmenbedingungen: Da die Vorgaben für die landwirtschaftliche Förderung im kommenden Förderzeitraum angepasst werden, muss auch der Rechner entsprechend aktualisiert werden. So ist sichergestellt, dass die Antragsteller jederzeit mit den aktuellen Berechnungsgrundlagen arbeiten.

Darüber hinaus wäre eine Erweiterung denkbar, die auch weitere Förderprogramme integriert, sodass nicht nur die Basisprämie, sondern auch zusätzliche Maßnahmen wie die Förderungen über die Öko-Regelungen direkt im System berechnet werden können. Auch eine detailliertere Ergebnisdarstellung in Form von Grafiken oder interaktiven Tabellen könnte die Benutzerfreundlichkeit weiter steigern.

Langfristig könnte der GAP-Prämienrechner zu einem umfassenden Planungsinstrument für landwirtschaftliche Betriebe ausgebaut werden, was nicht nur die aktuellen Prämien berechnet, sondern auch Prognosen für kommende Förderperioden ermöglicht.

Glossar

Begriff	Beschreibung
BitBucket	Ein verteiltes Versionskontrollsystem zur Nachverfolgung von Codeänderungen.
Confluence	Eine kollaborative Plattform von Atlassian zur Dokumentation und Wissensmanagement.
CSS	Eine Sprache zur Gestaltung und Layout von Webseiten
Gemeinsamer Antrag / GA	Das Antragsverfahren für Landwirte in Deutschland, um unterschiedliche Förder- und Ausgleichsmaßnahmen zu beantragen
Git-Repository	Ein zentraler Speicherort für den Code und die Dateien eines Projekts.
Java Bean	Eine Java Klasse die von Java Server Faces verwendet wird. Sie hat bestimmte Eigenschaften, um für Webseiten verwendet werden zu können.
Java Server Faces / JSF	Eine Java Framework für die Webentwicklung.
Jira	Ein Ticketverwaltungssystem für eine agile Softwareentwicklung.
JSON	Datenformat zur strukturierten Speicherung und Übertragung von Daten.
Nutzungscode / NC	Eine spezifische Codierung für eine bestimmte Fläche oder Maßnahme
Online-Antragstellung / OAS	Eine Webanwendung zum Stellen von Anträgen für Förderungen.
PostgreSQL	Ein freies, objektrelationales Datenbanksystem.
SCRUM	Ein Vorgehensmodell zur agilen Softwareentwicklung.
SpringBoot	Ein Framework zur Entwicklung von Java-Anwendungen, welches auf dem Spring-Framework basiert.
Tomcat	Ein Open-Source-Server für Java-Anwendungen.
Unit-Tests	Testen einer einzelnen abgeschlossenen Funktion, ohne den Rest der Anwendung.
Wasserfallmodell / erweitertes Wasserfallmodell	Eines der ältesten Entwicklungsmodelle. Es gibt verschiedene definierte Schritte, die eingehalten werden müssen. Das erweiterte Wasserfallmodell erlaubt es, einzelne Schritte zurückzugehen. Geeignet für Projekte mit geringem Umfang.
XHTML	Eine Kombination aus XML und HTML. Wird von Java Server Faces für Webseiten verwendet.

Abkürzungsverzeichnis

Begriff	Beschreibung
CSS	Cascading Style Sheets
DB	Datenbank
GA	Gemeinsamer Antrag
GAP	Gemeinsame Agrarpolitik der EU
JSF	Java Server Faces bzw. Jakarta Faces
JSON	JavaScript Object Notation
NC	Nutzungscode (Code für die Nutzung einer landwirtschaftlichen Fläche)
OAS	Online-Antragstellung
PI	Personen Ident
SQL	Structured Query Language
XHTML /HTML	eXtensible - / HyperText Markup Language

Anlageverzeichnis

A1: Bestätigung über die durchgeführte betriebliche Projektarbeit.....	16
A2: Protokoll über die betriebliche Projektarbeit.....	17
A3: Projekttagebuch.....	18
A4: Zeitaufwand für Prozessschritte.....	19
A5: Detaillierte Zeitplanung.....	20
A6: Detaillierte Ressourcenplanung.....	21
A7: Verwendetes Vorgehensmodell.....	22
A8: Jira Tickets zum GAP-Rechner.....	22
A9: Screenshot Excel-Rechner.....	23
A10: Ausschnitt aus der BasispraemieRepository-Klasse	24
A11: Sequenzdiagramm.....	25
A12: Programmablaufplan.....	26
A13: Ausschnitt aus der BasispraemieRechnerBean-Klasse	27
A14: Ausschnitt aus der BasispraemieRechnung-Klasse	28
A15: Klassendiagramm.....	29
A16: Screenshot Disclaimer.....	30
A17: Screenshot Ergebnis-Seite.....	30
A18: Ausschnitt aus der CSS-Datei	31
A19: Screenshot Entwicklerdokumentation Confluence.....	31

A1: Bestätigung über die durchgeführte betriebliche Projektarbeit**Anlage 1****Bestätigung über die durchgeführte betriebliche Projektarbeit**

Bitte füllen Sie die Anlagen 1 bis 4 in Druckschrift oder maschinell aus und fügen Sie diese der Projektdokumentation bei!

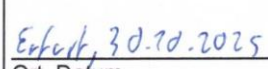
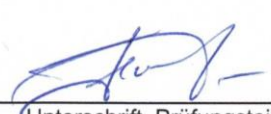
Antragsteller: Name: Havrysh Vorname: Mykola Straße: Am alten Backhaus 5 PLZ Ort: 99098 Erfurt Email: decrrier@ukr.net	Ausbildungsbetrieb (Praktikumsbetrieb): IBYKUS AG für Informationstechnologie Herman-Hollerith-Str. 1 99099 Erfurt
--	--

Projektbezeichnung (Auftrag/Teilauftrag):

"GAP-Prämienrechner für Direktzahlung in der Online-Antragstellung"

29.09.2025	13.10.2025	80
Durchführungsbeginn	Durchführungsende	Zeitaufwand in Stunden

Bestätigung des Ausbildungsbetriebes/Praktikumsbetriebes: Wir versichern, dass das Projekt, wie in der Dokumentation dargestellt, in unserem Unternehmen realisiert wurde.	
 Name, Vorname (Projektverantwortlicher)	 für Informationstechnologie Herman-Hollerith-Straße 1, 99099 Erfurt Stempel und Unterschrift (Unternehmen) E-Mail: info@ibykus.de

Verbindliche Erklärung des Prüfungsteilnehmers/der Prüfungsteilnehmerin: Ich versichere, durch meine Unterschrift, dass ich das Projekt und die dazugehörige Dokumentation selbständig und ohne fremde Hilfe angefertigt und alle Stellen, die ich wörtlich oder annähernd wörtlich aus Veröffentlichungen entnommen habe, als solche kenntlich gemacht habe. Die Arbeit hat in dieser Form keiner anderen Prüfungsinstitution vorgelegen.	
 Erfurt, 30.10.2025 Ort, Datum	 Unterschrift Prüfungsteilnehmer

A2: Protokoll über die betriebliche Projektarbeit



Industrie- und Handelskammer
Erfurt

Anlage 2

Protokoll über die betriebliche Projektarbeit

(vom Projektverantwortlichen im Unternehmen auszufüllen)

1. Arbeitszeit

- Die vom Prüfungsteilnehmer kalkulierte Zeit entspricht der betrieblichen Kalkulation
☒ ja ☐ nein

Wenn nein: Sie ist um% höher, um% niedriger

- Das Projekt wurde vom Prüfungsteilnehmer in der kalkulierten Zeit komplett fertiggestellt (einschließlich eventueller Nacharbeit):
☒ ja ☐ nein

Wenn nein: Um Std. früher fertig geworden

..... Std. länger gebraucht

2. Ausführung

- Wurde das Projekt entsprechend dem eingereichten Konzept ausgeführt?
☒ ja ☐ nein

Wenn nein: Begründung und Umfang der Hilfestellung:
.....
.....

- Das Projekt konnte ohne Nacharbeit im einwandfreien Zustand übergeben werden
☒ ja ☐ nein

Wenn nein: Begründung:

Umfang der Nacharbeit:

3. Dokumentation

Die Dokumentation wurde vom Prüfungsteilnehmer selbständig ohne fremde Hilfe erstellt.

☒ ja ☐ nein

Wenn nein: Worin bestehen die Abweichungen?

Wir bestätigen, dass die obigen Angaben wahrheitsgemäß sind.

30.10.2025

Datum

Unterschrift

Projektverantwortlicher

Unterschrift

Prüfungsteilnehmer

A3: Projekttagbuch

Anlage 3

Projekttagbuch

Datum	Tätigkeiten	Besonderheiten
29.09.25	Erstellung Zeitplan Ressourcenplanung Vorgehensmodell Festlegen Ist-Analyse Soll-Zustand-Analyse	
30.09.25	Nutzwertanalyse erstellt Entwicklerabsprachen Qualitätsanforderungen Tickets planen Schichtenmodell festlegen	Jira-Board eingerichtet
01.10.25	DTO festlegen Excel-Formeln für Prämien extrahieren und klären Repository-Schicht	Analyse der Excel-Formeln aufwendiger als erwartet
02.10.25	Datenquellen klären, Relevante Tabellenblätter identifizieren Entwurf der Geschäftslogik Basisprämie, Directzahlungen implementieren	
06.10.25	XHTML und CSS erstellen Tests	Erste Testberechnungen durchgeführt, Ergebnisse erfolgreich mit Excel verglichen.
07.10.25	Abnahme und Review durch Projektleiter und weitere Entwickler Fehlerbehebung und Optimierung Schreiben der Entwicklerdokumentation Beginn der Projektdokumentation	Rückmeldung des Projektleiters: kleinere Änderungen notwendig, wurden direkt umgesetzt.
08.10.25	Fortfhren mit der Projektdokumentation	
09.10.25	Fortfhren mit der Projektdokumentation	
10.10.25	Fertigstellen der Projektdokumentation	

A4: Zeitaufwand für Prozessschritte



Industrie- und Handelskammer
Erfurt

Anlage 4

Zeitaufwand für Prozessschritte

Durchführungsort	Tätigkeiten	benötigte Zeit in Std.
Erfurt - Ibykus Ag	Erstellung Zeitplan	1 St.
	Ressourcenplanung	1 St.
	Vorgehensmodell Festlegen	1 St.
	Ist-Analyse	1 St.
	Soll-Analyse	3 St.
	Wirtschaftlichkeitsanalyse	2 St.
	Nutzwertanalyse erstellt	1 St.
	Entwicklerabsprachen	2 St.
	Qualitätsanforderungen	1 St.
	Tickets planen	2 St.
	Excel-Formeln für Prämien extrahieren und klären	3 St.
	Datenquellen klären, Relevante Tabellenblätter identifizieren	3 St.
	Entwurf der Geschäftslogik	5 St.
	Fertigstellung Backend	18 St.
	XHTML und CSS erstellen	10 St.
	Tests	1 St.
	Abnahme und Review durch Projektleiter und weitere Entwickler	1 St.
	Fehlerbehebung und Optimierung	2 St.
	Schreiben der Entwicklerdokumentation	3 St.
	Projektdokumentation	19 St.
Summe		80 St.
Dokumentation der Tätigkeiten		22 St.
Gesamtzeit		80 St.

A5: Detaillierte Zeitplanung

Planung	3 St.
Erstellung Zeitplan	1 St.
Ressourcenplanung	1 St.
Vorgehensmodell Festlegen	1 St.
Analyse	12 St.
Ist-Zustand	1 St.
Soll-Zustand	3 St.
Wirtschaftlichkeitsanalyse	2 St.
Nutzwertanalyse erstellt	1 St.
Entwicklerabsprachen	2 St.
Qualitätsanforderungen	1 St.
Tickets planen	2 St.
Implementierung und Test	40 St.
Excel-Formeln für Prämien extrahieren und klären	3 St.
Datenquellen klären, Relevante Tabellenblätter identifizieren	3 St.
Entwurf der Geschäftslogik	5 St.
Umsetzung des Backend	18 St.
XHTML und CSS erstellen	10 St.
Test	1 St.
Abschluss des Projektes	3 St.
Abnahme und Review durch Projektleiter und weitere Entwickler	1 St.
Fehlerbehebung und Optimierung	2 St.
Schreiben der Projektdokumentation	22 St.
Schreiben der Entwicklerdokumentation	3 St.
Projektdokumentation	19 St.
Gesamt	80 St.

Tabelle 4: Detaillierte Zeitplanung

A6: Detaillierte Ressourcenplanung

Hardware
HP-Notebook
Tastatur
Maus
Headset
2 Monitore
Software
Confluence - Enterprise Wiki für die Dokumentation und Kommunikation von Wissen und den Wissensaustausch in Unternehmen
Gather Town - virtuelles Büro
Google Chrome - Browser
Jira - eine vielseitige webbasierte Software von Atlassian für das operative Projekt- und Aufgabenmanagement
Microsoft Excel - Erstellung von Tabellen
Microsoft Outlook - E-Mails, Termine, Kontakte
Microsoft Teams - Kommunikation-Software
Microsoft Word - Erstellung der Dokumentation
Windows 11 - Betriebssystem
BitBucket - Versionsverwaltung
Draw.io - Erstellung von Diagrammen für die Dokumentation
IntelliJ IDEA - eine integrierte Entwicklungsumgebung, in der das Projekt umgesetzt wurde
pgAdmin - Entwicklung, Administration und Pflege von PostgreSQL-Datenbanken
PostgreSQL - Open-Source-Datenbanksystem, das objektrelationale Daten speichert und verwaltet
Tomcat - Server auf dem die lokale Webanwendung und Datenbank läuft
Personalressourcen
Auszubildender
Softwareentwickler
Projektleiter

Tabelle 5: Detaillierte Ressourcenplanung

A7: Verwendetes Vorgehensmodell

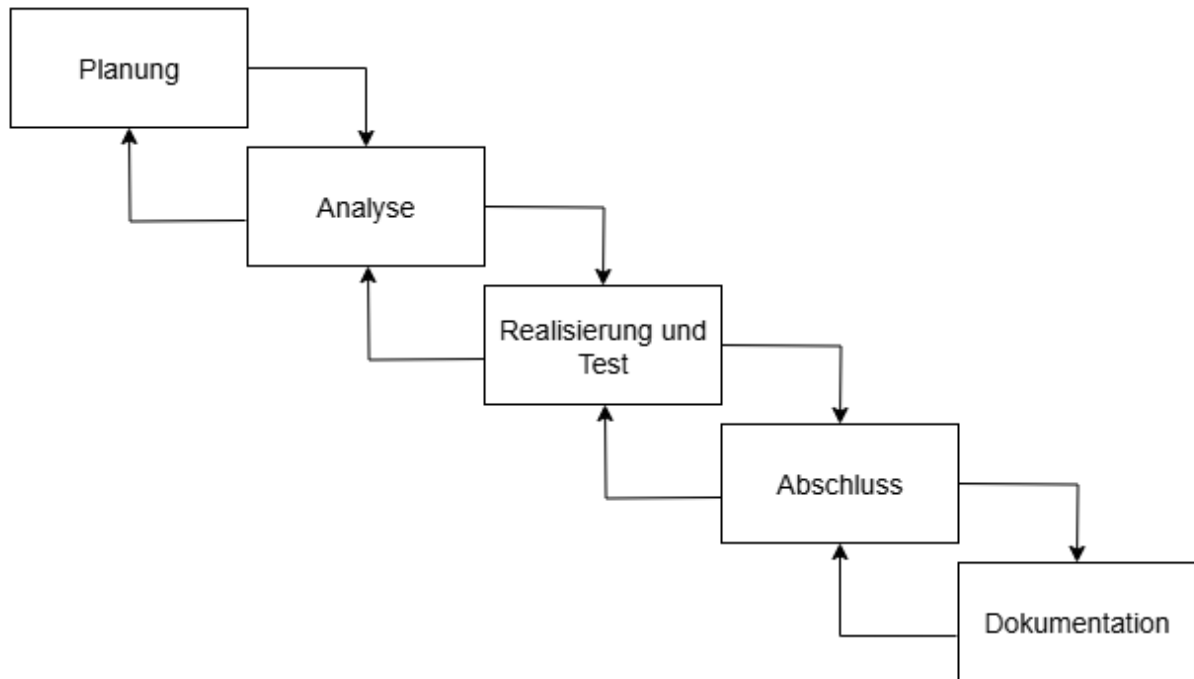


Abbildung 1: Verwendetes Vorgehensmodell

A8: Jira Tickets zum GAP-Rechner

☒	AB-25	Umverteilungsprämie-Rechnung implementieren	IMPLEMENTI...	FERTIG ▼	🔍
☒	AB-26	Juglandwirteprämie-Rechnung implementieren	IMPLEMENTI...	FERTIG ▼	🔍
☒	AB-27	Mutterkuhprämie-Rechnung implementieren	IMPLEMENTI...	FERTIG ▼	🔍
☒	AB-28	Schaf/Ziege-Prämie-Rechnung implementieren	IMPLEMENTI...	FERTIG ▼	🔍
☒	AB-30	Konstante in eine separate Klasse verschieben	IMPLEMENTI...	FERTIG ▼	🔍
☒	AB-31	Methode getValueNotNull	IMPLEMENTI...	FERTIG ▼	🔍
☒	AB-32	Ersetzen direkte DB-Abfrage. UniversalDBController benutzen!	IMPLEMENTI...	FERTIG ▼	🔍
☒	AB-33	Aktuelles Jahr in die SQL-Strings hinzufügen	IMPLEMENTI...	FERTIG ▼	🔍
☒	AB-34	DTO (Input, Result) festlegen	ARCHITEKTU...	FERTIG ▼	🔍
☒	AB-36	JSF-Tabelle für Direktzahlungen und für die allgemeinen Angaben	IMPLEMENTI...	FERTIG ▼	🔍
☒	AB-37	Zahlformatierung (de, 2 Nachkommastellen)	IMPLEMENTI...	FERTIG ▼	🔍

Abbildung 2: Jira Tickets

A9: Screenshot Excel-Rechner

Berechnung der möglichen Basisprämie ab 2025

(Achtung: Für die Richtigkeit der Berechnungen kann keine Gewähr übernommen werden.)

Angaben zur Betriebsstruktur				
Allgemeine Angaben		Betriebsname / Eigentümer/in:		
Wirtschaftsform	bitte wählen	Anschrift / Betriebssitz:		
Junglandwirt/in	bitte wählen			

Flächenausstattung	Tierhaltung (nur Raufutterfresser) (nötig für gekoppelte Tierprämie und ÖR 4)	Durchschnittsbestand (ganzjährig)	RGV Tierkategorie
Gesamte förderfähige Fläche 0,000 ha	bitte wählen	0	0,00
Ackerfläche gesamt (AF) 0,000 ha	bitte wählen	0	0,00
davon mit Gras oder Grünfutter (optional) 0,000 ha	bitte wählen	0	0,00
davon Ackerfutter mit Legumiosen (optional) 0,000 ha	bitte wählen	0	0,00
davon sonstige Ackerbrachen (optional; ohne Öko-Regelung 1a) 0,000 ha	bitte wählen	0	0,00
Grünlandfläche (DGL) 0,000 ha	bitte wählen	0	0,00
Dauerkulturen (DK) 0,000 ha	bitte wählen	0	0,00
	bitte wählen	0	0,00
Durchschnittliche RGV je ha DGL bei der Teilnahme an der Öko-Regelung 4 wird der durchschnittliche Viehbesatz des gesamten Jahres betrachtet.		0	
Betrieb gibt Kuhmilch oder -milcherzeugnisse ab		nein	

Aus den Angaben berechnete Direktzahlungen im Jahr 2025

2025			
		je Einheit	gesamt
Basisprämie [€/ha] (je ha landwirtschaftliche Fläche)		152	0
Umverteilungsprämie [€/ha]	1 - 40 ha LF	68	0
	41 - 60 ha LF	40	0
Junglandwirteprämie [€/ha]	max. 120 ha LF	134	0
gekoppelte Tierprämien	Mutterkuh, einmal gekalbt min. 3 Tiere	87	0
	♀ Schaf / Ziege, älter 10 Monate min. 6 Tiere	39	0
Summe Basisprämie		0,00 €	0 €

Abbildung 3: Screenshot Excel-Rechner

A10: Ausschnitt aus der BasispraemieRepository-Klasse

```
public Map<EBasisraemiePersonData, String> getPersonInfoById(String pi) { 1 usage  Havrysh Mykola *  
  
    Map<EBasisraemiePersonData, String> personInfo = new EnumMap<>(EBasisraemiePersonData.class);  
    String betriebsname = "";  
    String anschrift = "";  
  
    try {  
        List<JSONObject> personDataList = universalDBController.pull(ESQLQueries.PERSOENLICHE_DATEN, pi);  
        if (personDataList == null || personDataList.isEmpty()) {  
            return personInfo;  
        }  
        JSONObject personData = personDataList.get(0).getJSONObject( key: "CONTENT");  
        if (personData == null) {  
            return personInfo;  
        }  
  
        String persart = personData.optString( key: "PERSART", defaultValue: "");  
        String anrede = personData.optString( key: "ANREDE", defaultValue: "");  
        String vorname = personData.optString( key: "VORNAME", defaultValue: "");  
        String name = personData.optString( key: "NAME1", defaultValue: "");  
        String strasse = personData.optString( key: "STRASSE", defaultValue: "");  
        String plz = personData.optString( key: "PLZ", defaultValue: "");  
        String ort = personData.optString( key: "ORT", defaultValue: "");  
  
        if (persart != null && persart.toLowerCase().equals("natürlich")) {  
            betriebsname = String.join( delimiter: " ", anrede, vorname, name);  
        }  
        if (persart != null && persart.toLowerCase().equals("juristisch")) {  
            betriebsname = name;  
        }  
    }  
}
```

Abbildung 4: BasispraemieRepository-Klasse

A11: Sequenzdiagramm

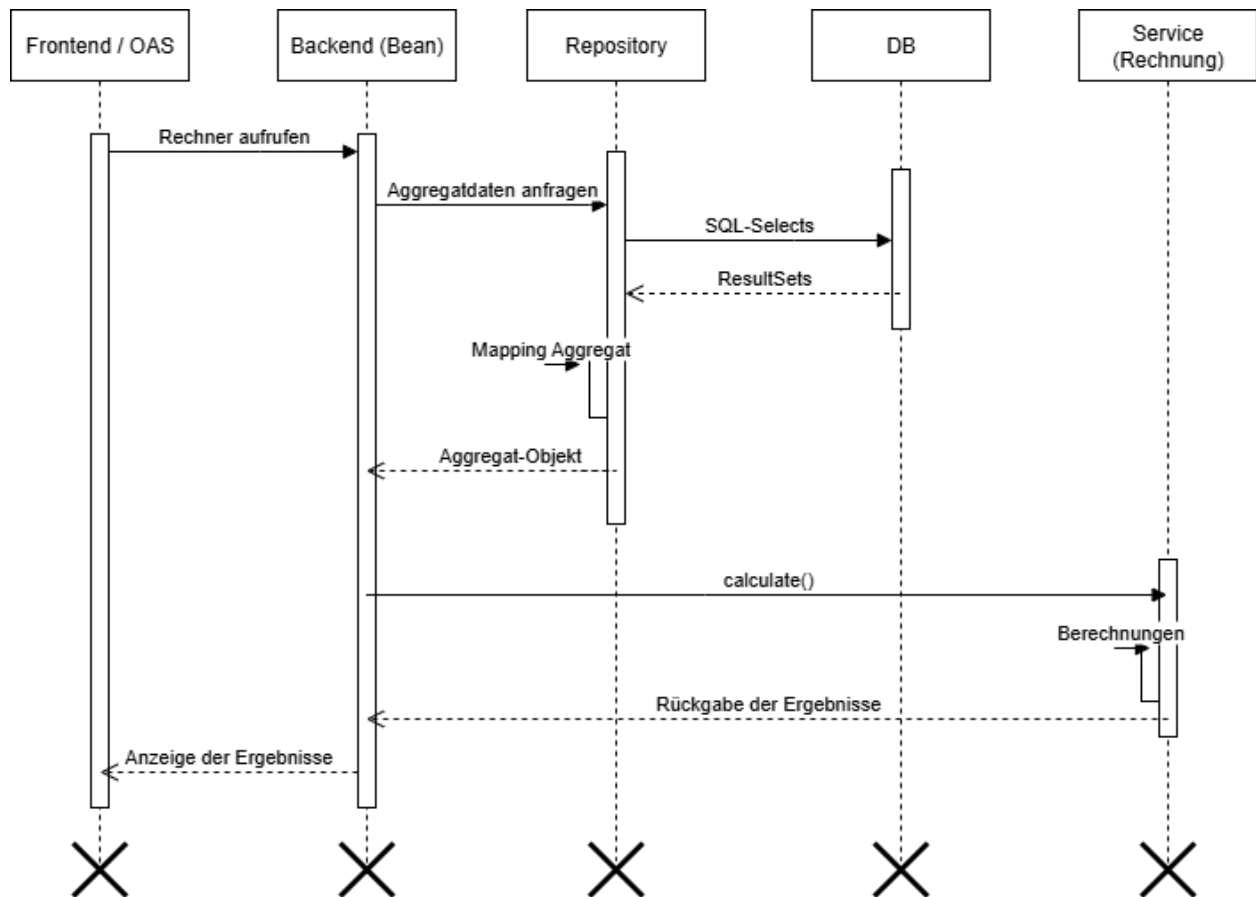


Abbildung 5: Sequenzdiagramm

A12: Programmablaufplan

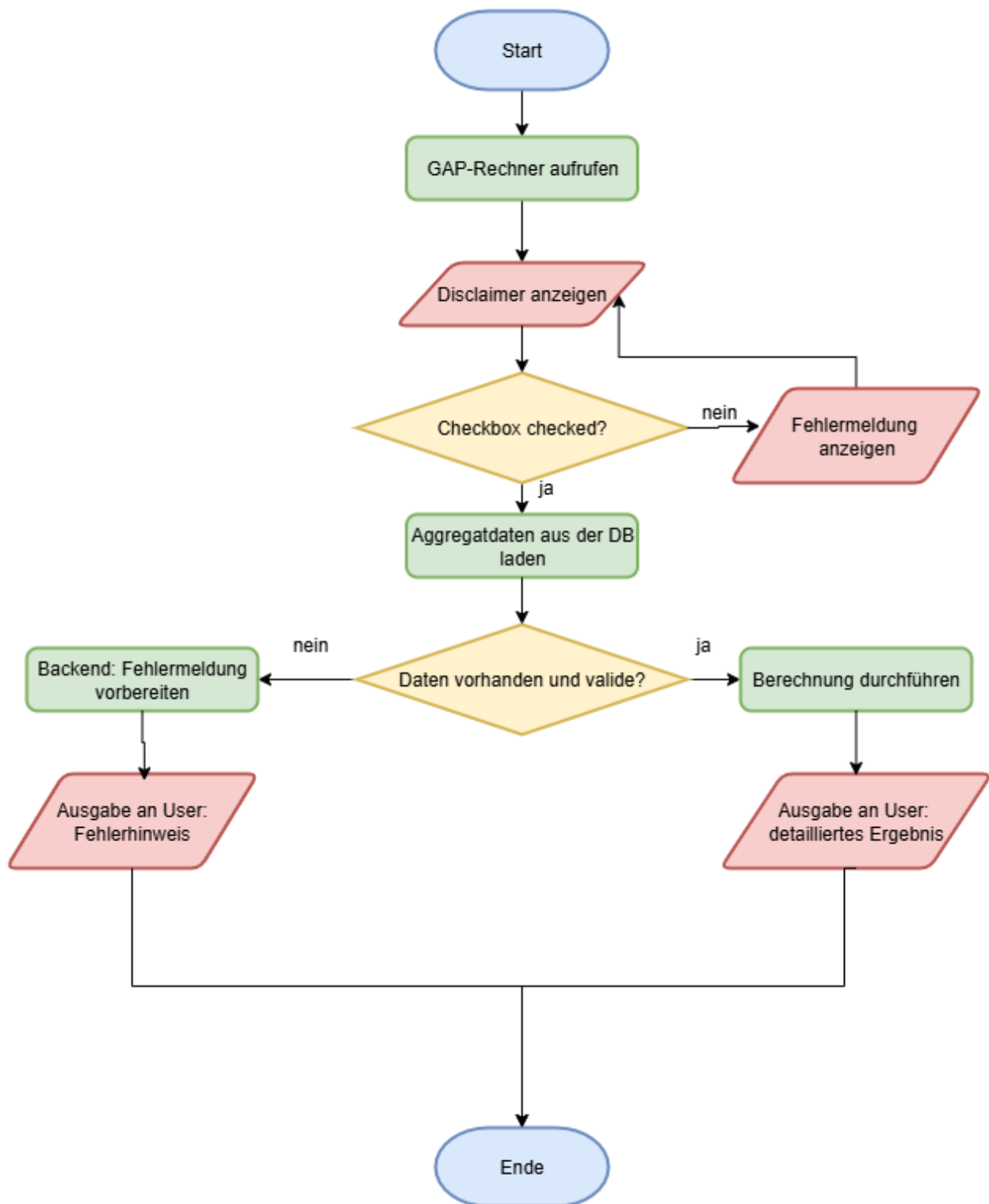


Abbildung 6: Programmablaufplan

A13: Ausschnitt aus der BasispraemieRechnerBean-Klasse

```
@PostConstruct  🧑 Havrysh Mykola
public void init() {
    try {
        String pi = sessionBean.getOasxUser().getPI();
        BasispraemieAggregatInfo aggregatInfo = repository.loadAllForPi(pi);

        // Flächen
        this.afHa = aggregatInfo.getAfHa();
        this.dglHa = aggregatInfo.getDglHa();
        this.dkHa = aggregatInfo.getDkHa();
        this.flaecheGesamt = afHa.add(dglHa).add(dkHa);

        // Persönliche Daten
        this.betriebsname = aggregatInfo.getBetriebsname();
        this.anschrift = aggregatInfo.getAnschrift();

        // Flags
        this.wirtschaftsform = aggregatInfo.isOekologisch();
        this.junglandwirt = aggregatInfo.isJunglandwirt();
        this.gibtKuhmilchAb = aggregatInfo.isGibtKuhmilchAb();

        // GA DATA DIREKT
        Map<EBasispraemieTiere, BigDecimal> td = aggregatInfo.getTierData();
        this.tierData.clear();
        this.tierData.putAll(td);

        // GA GES
        Map<String, BigDecimal> gd = aggregatInfo.getGesData();
        this.gesData.clear();
        this.gesData.putAll(gd);

        this.basispraemieResult = BasispraemieRechnung.calculate(
            this.flaecheGesamt,
            this.junglandwirt,
            this.gibtKuhmilchAb,
            this.gesData
        );
    }
}
```

Abbildung 7: BasispraemieRechnerBean-Klasse

A14: Ausschnitt aus der BasispraemieRechnung-Klasse

```
public static BasispraemieResult calculate(  Havrysh Mykola *
    BigDecimal flaecheGesamt,
    boolean isJunglandwirt,
    boolean gibtKuhMilchAb,
    Map<String, BigDecimal> gesData) {

    gesData = Optional.ofNullable(gesData).orElse(new HashMap<>());
    BigDecimal basisBetrag = getValueNotNull(flaecheGesamt
        .multiply(BASIS_EUR_HA)
        .setScale(EUR_SCALE, RoundingMode.HALF_UP));

    BigDecimal umverteilungspraemie = calcUmverteilungsPraemie(flaecheGesamt);
    BigDecimal junglandwirtepraemie = calcJunglandwirtePraemie(flaecheGesamt, isJunglandwirt);
    BigDecimal mutterkueheAnzahl = gesData.getOrDefault( key: "BEANTRAGTETIERE", BigDecimal.ZERO);
    BigDecimal szAnz = gesData.getOrDefault( key: "BEANTRAGTESZTIERE", BigDecimal.ZERO);
    BigDecimal mutterkuhPraemie = calcMutterkuhPraemie(mutterkueheAnzahl, gibtKuhMilchAb);
    BigDecimal schafZiegePraemie = calcSchafZiegePraemie(szAnz);

    BigDecimal summe = basisBetrag.add(umverteilungspraemie).add(junglandwirtepraemie)
        .add(mutterkuhPraemie).add(schafZiegePraemie)
        .setScale(EUR_SCALE, RoundingMode.HALF_UP);

    return new BasispraemieResult(
        BASIS_EUR_HA,
        UMV_1_40_EUR_HA,
        UMV_41_60_EUR_HA,
        JLW_EUR_HA,
        MUTTERKUH_EUR_TIER,
        SZ_EUR_TIER,
        basisBetrag, umverteilungspraemie, junglandwirtepraemie,
        mutterkuhPraemie, schafZiegePraemie, summe
    );
}
```

Abbildung 8: BasispraemieRechnung-Klasse

A15: Klassendiagramm

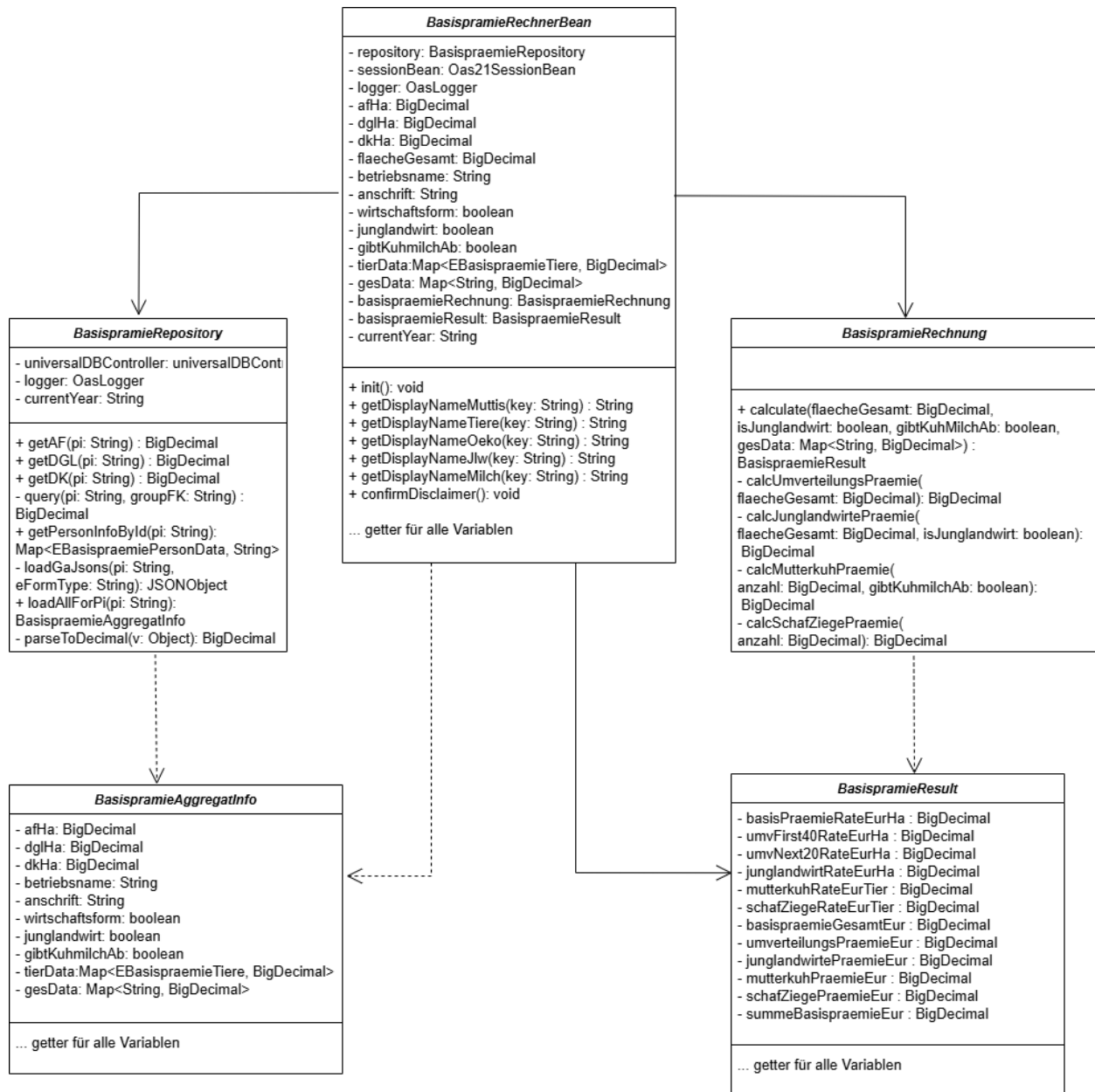


Abbildung 9: Klassendiagramm

A16: Screenshot Disclaimer

Wichtiger Hinweis

Der Prämienrechner dient nur als Hilfestellung und erste Orientierung bei der Beantragung von Fördermaßnahmen. Es kann nicht ausgeschlossen werden, dass es noch zu Änderungen der Förderungsvoraussetzungen kommt. Daher erfolgen alle Berechnungen ohne Gewähr. Die zur Berechnung verwendeten Daten stammen aus dem Gemeinsamen Antrag. Hierzu wurden Informationen aus den Formularen "Direktzahlung", "Gekoppelte Einkommensstützung" und Angaben aus dem "Flächen- und Nutzungsnachweis" verwendet.

☒ Ich habe den Hinweis gelesen und stimme zu

[Weiter](#)

Abbildung 10: Screenshot Disclaimer

A17: Screenshot Ergebnis-Seite

Prämienrechner

Aus den Angaben berechnete Direktzahlungen im Jahr 2025		
Position	je Einheit	Gesamt
Basisprämie [€/ha]	152,00 €	8.379,76 €
Umverteilungsprämie [€/ha]	1-40 ha: 68 €, 41-60 ha: 40 €	3.325,20 €
Junglandwirteprämie [€/ha] (max. 120 ha LF)	134,00 €	7.387,42 €
Tierprämien: Mutterkühe [€/Tier] (min. 3 Tiere und keine Milchabgabe)	87,00 €	0,00 €
Tierprämien: Mutterschafe/ Mutterziegen [€/Tier] (min. 6 Tiere)	39,00 €	0,00 €
Summe		19.092,38 €

Angaben zur Betriebsstruktur	
Allgemeine Angaben	
Betriebsname / Eigentümer/in:	Max Mustermann
Wirtschaftsform	Ökologisch
Junglandwirt/in	Ja
Betrieb gibt Kuhmilch oder -milcherzeugnisse ab:	Ja

Aufteilung der landwirtschaftlich genutzten Fläche	
Ackerfläche	2,788 ha
Grünlandfläche	52,342 ha
Dauerkulturen	0,000 ha
Gesamte förderfähige Fläche	55,130 ha

Tierübersicht für gekoppelte Einkommensstützung	
Mutterkühe	21
Mutterschafe/Mutterziegen	0

Abbildung 11: Screenshot Ergebnis-Seite

A18: Ausschnitt aus der CSS-Datei

```
body {
    background: #f7f7f7 !important;
}

.blueheader{
    background-color:#0082b4 !important;
    color:white !important;
    font-weight:bold !important;
    font-size: 20px; !important;
    text-align:center; !important;
}

table thead tr th:last-child{
    width: 150px;
    max-width: 150px;
}

.container-title{
    position: relative;
    background: #B9CE41;
    color: white;
    margin-bottom: -30px;
    z-index: 2;
    width: 400px;
    border-radius: 40px 0;
```

Abbildung 12: CSS-Datei

A19: Screenshot Entwicklerdokumentation Confluence

Backend

Dateiname	Methodenname	Beschreibung
BasispraemieRepository.java	ist für die Beschaffung der für die Basisprämie benötigten Informationen aus der Datenbank zuständig, enthält Methoden zur Wertauswahl (z. B. nach Personendaten, Fläche, Tieren oder anderen Parametern),	\\src\\main\\java\\de\\ibykus\\oasx\\controller\\basispraemie
	Methoden	
	getAF(), getDGL(), getDK()	geben die Summe der Flächen bestimmter Kategorien zurück
	query()	führt die Datenauswahl für Methoden getAF(), getDGL(), getDK() durch
	getPersonInfoByID()	aggregiert die persönliche Informationen
	loadAllForPi()	Aggregiert alle Daten in einem Objekt zur weiteren Übertragung
	loadGaJsons()	führt die Datenauswahl für loadAllForPi() durch
	parseToDecimal()	konvertiert String in BigDecimal
	getValueNotNull()	Vermeidung von NullPointerException
BasisPraemieAggregatInfo.java	speichert gesammelte Daten zur Basisprämie, dient als „Container“ für Werte, damit diese bequem weiter an den Rechner übertragen werden können.	\\src\\main\\java\\de\\ibykus\\oasx\\controller\\basispraemie\\dto
BasisPraemieRechnerBean.java	empfängt die Daten, aggregiert die Informationen mithilfe von BasisPrämieAggregatInfo und gibt das Ergebnis der Berechnung zur Anzeige im Frontend zurück	\\src\\main\\java\\de\\ibykus\\oasx\\view\\beans

Abbildung 13: Screenshot Entwicklerdokumentation Confluence

GAP-Prämienrechner

Entwicklung eines Rechners für die Berechnung der Direktzahlung in der Online-Antragstellung



Quellenverzeichnis

Über Ibykus:

<https://www.ibykus.de/unternehmensprofil/> (30.09.2025)

GAP-Prämienrechner

<https://llh.hessen.de/unternehmensfuehrung/agrarpolitik-und-foerderung/direktzahlungen/gap-pramienrechner/>

XHTML

<https://www.w3.org/TR/xhtml1/>

Eidesstattliche Erklärung

Ich versichere an Eides statt, dass ich die Arbeit selbstständig und ohne Benutzung anderer als der angegebenen Hilfsmittel angefertigt habe. Alle Stellen, die wörtlich oder sinngemäß aus Veröffentlichungen entnommen sind, habe ich als solche gekennzeichnet.

Ort, Datum

Unterschrift